Sorting Strategies

Sorting algorithms are fundamental to computer science. Here are some of the most common ones.

Bubble Sort

Bubble Sort is a simple algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. It's easy to implement but inefficient for large datasets.

- **Time Complexity**: O(n2)
- **Space Complexity**: O(1)

Java

```
void bubbleSort(int arr[]) {
  int n = arr.length;
  for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
      if (arr[j] > arr[j + 1]) {
        // swap arr[j+1] and arr[j]
        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
      }
    }
  }
}
```

Selection Sort

This algorithm divides the input list into two parts: a sorted sublist and an unsorted sublist. It repeatedly finds the minimum element from the unsorted sublist and moves it to the sorted sublist.

- **Time Complexity**: O(n2)
- **Space Complexity**: O(1)

Java

```
void selectionSort(int arr[]) {
  int n = arr.length;
  for (int i = 0; i < n - 1; i++) {
    // Find the minimum element in unsorted array
    int min_idx = i;
    for (int j = i + 1; j < n; j++) {
      if (arr[j] < arr[min_idx]) {
        min_idx = j;
      }
    }
    // Swap the found minimum element with the first element
    int temp = arr[min_idx];
    arr[min_idx] = arr[i];
    arr[i] = temp;
  }
}
```

Merge Sort

Merge Sort is a **divide-and-conquer** algorithm. It divides the array into two halves, calls itself for the two halves, and then merges the two sorted halves. It's much more efficient than Bubble or Selection Sort. 🚀

- **Time Complexity**: O(nlogn)
- **Space Complexity**: O(n)

Java

```
void merge(int arr[], int l, int m, int r) {
   int n1 = m - l + 1;
   int n2 = r - m;

   int L[] = new int[n1];
   int R[] = new int[n2];

   for (int i = 0; i < n1; ++i) L[i] = arr[l + i];
   for (int j = 0; j < n2; ++j) R[j] = arr[m + 1 + j];

   int i = 0, j = 0, k = l;
   while (i < n1 && j < n2) {
      if (L[i] <= R[j]) {
         arr[k++] = L[i++];
      } else {
         arr[k++] = R[j++];
      }
   }

   while (i < n1) arr[k++] = L[i++];
   while (j < n2) arr[k++] = R[j++];
}
```

```
void mergeSort(int arr[], int l, int r) {
  if (l < r) {
    int m = l + (r - l) / 2;
    mergeSort(arr, l, m);
    mergeSort(arr, m + 1, r);
    merge(arr, l, m, r);
  }
}
```

Quick Sort

Quick Sort is also a **divide-and-conquer** algorithm. It picks an element as a **pivot** and partitions the given array around the picked pivot. It's generally faster in practice than Merge Sort.

- **Time Complexity**: Average case O(nlogn), Worst case O(n2)
- **Space Complexity**: O(logn)

Java

```
int partition(int arr[], int low, int high) {
  int pivot = arr[high];
  int i = (low - 1); // index of smaller element
  for (int j = low; j < high; j++) {
    if (arr[j] < pivot) {
      i++;
      // swap arr[i] and arr[j]
      int temp = arr[i];
      arr[i] = arr[j];
      arr[j] = temp;
    }
```

```
    }
    // swap arr[i+1] and arr[high] (or pivot)
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

DSA Questions

Here are some common DSA questions with solutions.

---

1. Reverse a String

This is a classic warm-up question. The goal is to reverse a given string without using built-in reverse functions.

Java

```
public String reverseString(String str) {
    char[] charArray = str.toCharArray();
    int left = 0, right = charArray.length - 1;
```

```
    while (left < right) {
        // Swap characters
        char temp = charArray[left];
        charArray[left] = charArray[right];
        charArray[right] = temp;
        left++;
        right--;
    }
    return new String(charArray);
}

// Example usage:
// reverseString("hello") -> "olleh"
```

---

2. Check for Palindrome

A palindrome is a word, phrase, or sequence that reads the same backward as forward.

Java

```
public boolean isPalindrome(String str) {
    int left = 0;
    int right = str.length() - 1;
    while (left < right) {
        if (str.charAt(left) != str.charAt(right)) {
            return false;
        }
        left++;
        right--;
```

```
    }
    return true;
}

// Example usage:
// isPalindrome("madam") -> true
// isPalindrome("world") -> false
```

---

3. Find the Middle of a Linked List

This can be solved efficiently using the **two-pointer** (or "runner") technique. One pointer moves one step at a time (slow), while the other moves two steps at a time (fast). When the fast pointer reaches the end, the slow pointer will be at the middle.

Java

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int x) { val = x; }
}

public ListNode findMiddle(ListNode head) {
    if (head == null) {
        return null;
    }
    ListNode slow = head;
    ListNode fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
```

```
    fast = fast.next.next;
  }
  return slow;
}
```

Design Patterns

Design patterns are reusable solutions to commonly occurring problems within a given context in software design. 🧐

Singleton Pattern

Ensures a class has **only one instance** and provides a global point of access to it. It's useful for managing shared resources like a database connection or a logger.

Java

```
public class Singleton {
  // The sole instance of the class.
  private static Singleton instance;

  // Private constructor to prevent instantiation from other classes.
  private Singleton() {}

  // Public method to provide access to the instance.
  public static Singleton getInstance() {
    if (instance == null) {
      instance = new Singleton();
    }
    return instance;
```

```
    }

    public void showMessage() {
        System.out.println("Hello from Singleton!");
    }
}
```

Factory Pattern

Creates objects without exposing the instantiation logic to the client. It refers to the newly created object through a common interface.

Java

```
// Interface for a product
interface Shape {
    void draw();
}

// Concrete products
class Circle implements Shape {
    @Override
    public void draw() { System.out.println("Drawing a Circle"); }
}

class Rectangle implements Shape {
    @Override
    public void draw() { System.out.println("Drawing a Rectangle"); }
}
```

```
// Factory class
class ShapeFactory {
   public Shape getShape(String shapeType) {
      if (shapeType == null) {
         return null;
      }
      if (shapeType.equalsIgnoreCase("CIRCLE")) {
         return new Circle();
      } else if (shapeType.equalsIgnoreCase("RECTANGLE")) {
         return new Rectangle();
      }
      return null;
   }
}
```

Java 8 Features

Java 8 introduced many significant features that modernized the language. ✨

Lambda Expressions

A lambda expression is a short block of code that takes in parameters and returns a value. They are similar to methods but don't need a name. They are often used to implement functional interfaces.

Java

```
// Old way: Anonymous class
new Thread(new Runnable() {
```

```
    @Override
    public void run() {
        System.out.println("Thread running!");
    }
}).start();

// New way: Lambda expression
new Thread(() -> System.out.println("Thread running with Lambda!")).start();
```

Streams API

The Streams API is used to process collections of objects in a functional style. A stream is a sequence of elements that supports various methods that can be pipelined to produce the desired result.

Java

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

List<String> words = Arrays.asList("apple", "banana", "cherry", "date");

// Filter words that start with 'a', convert to uppercase, and collect to a new list.
List<String> result = words.stream()
    .filter(s -> s.startsWith("a"))
    .map(String::toUpperCase)
    .collect(Collectors.toList());

System.out.println(result); // Output: [APPLE]
```

Optional Class

Optional is a container object that may or may not contain a non-null value. It was introduced to avoid NullPointerException.

Java

```
import java.util.Optional;


String name = null;

Optional<String> opt = Optional.ofNullable(name);


// Check if a value is present

if (opt.isPresent()) {

   System.out.println(opt.get());

} else {

   System.out.println("Name is not present.");

}


// A more elegant way to handle the absence of a value

System.out.println(opt.orElse("Default Name")); // Output: Default Name
```

OOPs Concepts

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects".

Encapsulation

Encapsulation is the bundling of **data (attributes)** and the **methods (functions)** that operate on the data into a single unit or class. It restricts direct access to some of an object's components, which is a means of preventing unintended interference. 🔐

Java

```
class Person {
  // private data members
  private String name;
  private int age;

  // public getter and setter methods
  public String getName() {
    return name;
  }

  public void setName(String name) {
    this.name = name;
  }

  public int getAge() {
    return age;
  }

  public void setAge(int age) {
    if (age > 0) { // Data validation
      this.age = age;
    }
```

```
    }
}
```

Inheritance

Inheritance is a mechanism wherein a new class derives properties and behaviors (methods) from an existing class. The class that is inherited from is called the **superclass** (or parent), and the class that inherits is called the **subclass** (or child). It promotes code reusability. 👨‍👩‍👧

Java

```
// Superclass
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

// Subclass
class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}

// Usage:
// Dog myDog = new Dog();
// myDog.eat();  // Inherited from Animal
// myDog.bark(); // Defined in Dog
```

Polymorphism

Polymorphism (from Greek, meaning "many forms") allows methods to do different things based on the object it is acting upon. It is mainly achieved through **method overriding** (runtime polymorphism) and **method overloading** (compile-time polymorphism).

Java

```
// Method Overriding
class Shape {
  void draw() {
    System.out.println("Drawing a shape.");
  }
}

class Circle extends Shape {
  @Override // Annotation to indicate method overriding
  void draw() {
    System.out.println("Drawing a circle.");
  }
}

// Method Overloading
class Calculator {
  int add(int a, int b) {
    return a + b;
  }

  double add(double a, double b) {
```

```
    return a + b;
  }
}
```

Abstraction

Abstraction means hiding complex implementation details and showing only the essential features of the object. It is achieved using **abstract classes** and **interfaces**.

Java

```
// Abstract class
abstract class Vehicle {
  // Abstract method (does not have a body)
  public abstract void startEngine();

  // Regular method
  public void stopEngine() {
    System.out.println("Engine stopped.");
  }
}

class Car extends Vehicle {
  @Override
  public void startEngine() {
    System.out.println("Car engine started.");
  }
}
```

```
// Interface
interface Drawable {
   void draw(); // Abstract by default
}

class Square implements Drawable {
   public void draw() {
      System.out.println("Drawing a square.");
   }
}
```